



AMERICAN INTERNATIONAL UNIVERSITY-BANGLADESH

Faculty of Engineering

Lab Report

Experiment # 04

Experiment Title: Familiarization with a microcontroller, the study of blink test and implementation of a traffic control system using microcontrollers.

Date of Perform:	2 November 2025	Date of Submission:	5 November 2025
Course Title:	MICROPROCESSOR AND EMBEDDED SYSTEMS LAB		
Course Code:	EEE 4103	Section:	[O]
Semester:	Fall 2025-26	Degree Program:	BSc in CSE
Course Teacher:	Dr. Md. Rukonuzzaman		

Declaration and Statement of Authorship:

1. I/we hold a copy of this Assignment/Case Study, which can be produced if the original is lost/damaged.
2. This Assignment/Case Study is my/our original work and no part of it has been copied from any other student's work or from any other source except where due acknowledgment is made.
3. No part of this Assignment/Case Study has been written for me/us by any other person except where such collaboration has been authorized by the concerned teacher and is acknowledged in the assignment.
4. I/we have not previously submitted or currently submitting this work for any other course/unit.
5. This work may be reproduced, communicated, compared, and archived to detect plagiarism.
6. I/we permit a copy of my/our marked work to be retained by the Faculty Member for review by any internal/external examiners.
7. I/we understand that Plagiarism is the presentation of the work, idea, or creation of another person as though it is your own. It is a form of cheating and is a very serious academic offense that may lead to expulsion from the University. Plagiarized material can be drawn from, and presented in, written, graphic, and visual forms, including electronic data, and oral presentations. Plagiarism occurs when the origin of the source is not appropriately cited.
8. I/we also understand that enabling plagiarism is the act of assisting or allowing another person to plagiarize or copy my/our work.

** Student(s) must complete all details except the faculty use part.*

*** Please submit all assignments to your course teacher or the office of the concerned teacher.*

Group # 06

Sl No	Name	ID	PROGRAM	SIGNATURE
1	Chinmoy Guha	22-48056-2	BSc in CSE	
2	Nafiur Rahman Siam	23-50834-1	BSc in CSE	
3	Suvra Chakraborty	22-48067-2	BSc in CSE	
4	Mohammad Ansar Uddin	22-47975-2	BSc in CSE	
5	Arfanul Hoque Nehal	23-54270-3	BSc in CSE	
6	Mashkur Ibtesham Wazed	23-54266-3	BSc in CSE	

<i>Faculty use only</i>		
FACULTY COMMENTS	Marks Obtained	
	Total Marks	

Table of Contents

Experiment Title	3
Objectives	3
Equipment List	3
Circuit Diagram	4
Code/Program	5
Hardware Output Results	6
Experimental Output Results	7
Simulation Output Results	8
Answers to the Questions in the Lab Manual	9
Discussion	12
References	13

Experiment Title: Study of a Digital Timer using the millis() function of Arduino to avoid problems associated with the delay () function and power measurement in different power reduction modes while the circuit is in sleep modes..

Introduction

- To explore the operation and application of the millis() function in Arduino programming.
- To design and develop a digital timer system that sequentially activates multiple LEDs at one-minute intervals using an Arduino microcontroller.
- To monitor and measure the runtime of a project using Arduino's built-in timing capabilities.
- To implement a controlled LED sequence triggered by a user-operated switch in combination with the Arduino board.

Theory and Methodology

In basic Arduino projects, the delay() function is often used to create time intervals. While effective for simple tasks, delay() halts the entire program during the wait period, meaning the Arduino cannot perform other operations like checking inputs or updating outputs during that time. This becomes a major limitation in projects requiring multitasking or real-time responsiveness.

To overcome this issue, the millis() function is used. millis() returns the number of milliseconds that have passed since the Arduino was powered on or reset. Unlike delay(), it doesn't pause the program, allowing the Arduino to continue running other code simultaneously.

However, storing the value returned by millis() requires an appropriate data type. The commonly used int data type (16-bit) ranges only from -32,768 to 32,767, which can overflow in less than a minute when tracking time in milliseconds. Instead, the unsigned long type (32-bit) is used, which stores only positive values and can count up to 4,294,967,295 milliseconds—approximately 50 days—before resetting. This makes it ideal for tracking time over longer periods.

In this timer project, a tilt switch is used as a user input to trigger the LED sequence. A tilt switch functions like a button but responds to movement or angle. Inside the switch is a small metal ball that rolls and completes the circuit when tilted to a specific angle, which the Arduino reads as a HIGH or LOW signal. The system is designed to light up six LEDs in sequence, with a one-minute interval between each activation. Once triggered, the first LED lights up, followed by the second after one minute, continuing until all six LEDs are on. The total sequence runs for six minutes, demonstrating both timed control and input-triggered behavior using millis().

Objectives:

To become familiar with microcontroller programming by performing a basic blink test and implementing a traffic light control system using Arduino, focusing on timer-based control and external interrupts for real-time system behavior.

Equipment List:

- 1) Arduino board)
- 2) Arduino Microcontroller board
- 3) LED lights (Red, Green, and Yellow- each 2)
- 4) Six 100 Ω resistors and one 10 k Ω resistor
- 5) One tilt switch
- 6) Jumper wires

Circuit Diagram:

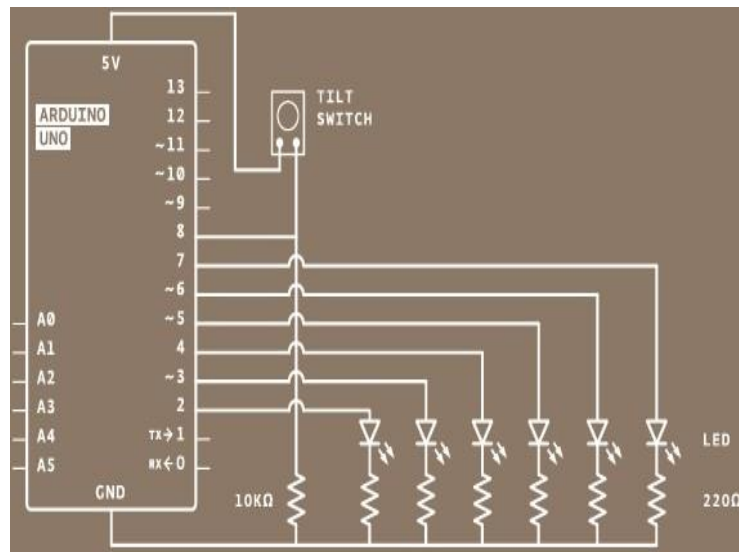
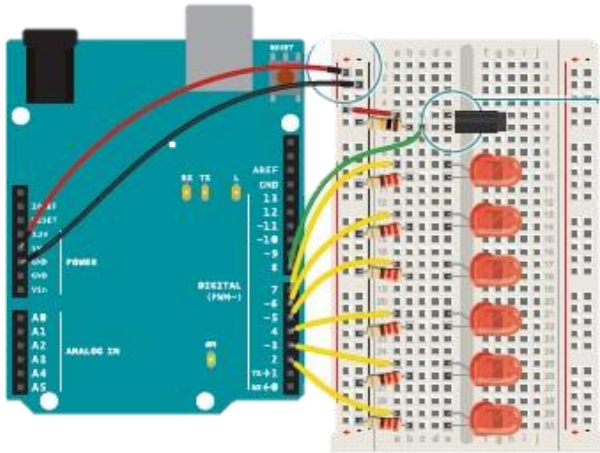


Figure-1: Circuit Diagram for LED Light Pattern Control System.

Code/Program

```
LED_PatternControl
}
pinMode(SwitchPin, INPUT);
}

void loop() {
  unsigned long CurrentTime = millis();
  if (CurrentTime - PreviousTime > interval) {
    PreviousTime = CurrentTime;

    digitalWrite(led, HIGH);
    led++;
    if (led == 7){
    }
    } SwitchState = digitalRead(SwitchPin);
    if (SwitchState != PrevSwitchState){
    for (int x = 2 ; x < 8; x++) {
      digitalWrite(x, LOW);
    }
    led = 2;
    PreviousTime = CurrentTime;
  }
  PrevSwitchState = SwitchState;
}
```

Figure 2: code for the LED Light Pattern Control System part a.
Figure 3: code for the LED Light Pattern Control System part b.

Code/Program:

The following is the code for the implementation of the digital timer using The millis () test with the necessary code explanation:

```
const int SwitchPin = 8;
unsigned long PreviousTime = 0;
int SwitchState = 0;
int PrevSwitchState = 0;
int led = 2;
long interval = 4000; // for 1 min = 60,000 ms delay
void setup() {
  for (int x = 2; x < 8; x++) {
    pinMode(x, OUTPUT);
  }
  pinMode(SwitchPin, INPUT);
}
void loop() {
```



```

unsigned long CurrentTime = millis();
if (CurrentTime - PreviousTime > interval) {
  PreviousTime = CurrentTime;
  digitalWrite(led, HIGH);
  led++;
}
SwitchState = digitalRead(SwitchPin);
if (SwitchState != PrevSwitchState){
  for (int x = 2 ; x < 8; x++) {
    digitalWrite(x, LOW);
  }
  led = 2;
  PreviousTime = CurrentTime;
}
PrevSwitchState = SwitchState;
}

```

Code Explanation:

This Arduino code is designed to create a digital timer using the millis() function, which avoids halting program execution, unlike the delay() function. Below is a breakdown of each section of the code with its explanation:

const int SwitchPin = 8;

Declares a constant integer variable named SwitchPin and assigns it the value 8. This pin is connected to a switch.

unsigned long PreviousTime = 0;

Declares an unsigned long variable PreviousTime initialized to 0. It is used to store the timestamp from the previous cycle.

int SwitchState = 0;

Declares an integer variable SwitchState with an initial value of 0. It holds the current state of the switch.

int PrevSwitchState = 0;

Declares an integer variable PrevSwitchState, also initialized to 0, to store the switch's previous state.

int led = 2;

Declares an integer variable led with the value 2. This indicates the pin controlling the LED.

long interval = 4000;

A long integer variable interval is defined with the value 4000, representing a 4-second time interval in milliseconds.

void setup()

The setup function runs once and is used to configure initial settings.

for (int x = 2; x < 8; x++) { pinMode(x, OUTPUT); }

A for loop is used to set pins 2 through 7 as output pins.

pinMode(SwitchPin, INPUT);

Configures pin 8 (the switch pin) as an input.

void loop()

The main loop of the program where continuous execution occurs.

unsigned long CurrentTime = millis();

Captures the current time in milliseconds since the program started and stores it in CurrentTime.
if (CurrentTime - PreviousTime > interval)
Checks if the time passed since the last recorded PreviousTime exceeds the defined interval (4 seconds).
PreviousTime = CurrentTime;
Updates PreviousTime with the latest value of CurrentTime.
digitalWrite(led, HIGH);
Turns on the LED connected to the pin specified by the variable led.
SwitchState = digitalRead(SwitchPin);
Reads the current state of the switch and stores it in SwitchState.
if (SwitchState != PrevSwitchState)
Checks whether the current state of the switch has changed compared to its previous state.
for (int x = 2; x < 8; x++) { digitalWrite(x, LOW); }
Turns off all LEDs connected to pins 2 through 7 by setting them to LOW.
led = 2;
Resets the LED pin variable to its starting value (pin 2).
PreviousTime = CurrentTime;
Updates the PreviousTime again to reflect the time when the switch state changed.
PrevSwitchState = SwitchState;
Updates PrevSwitchState to match the current SwitchState.

Hardware Output Results:

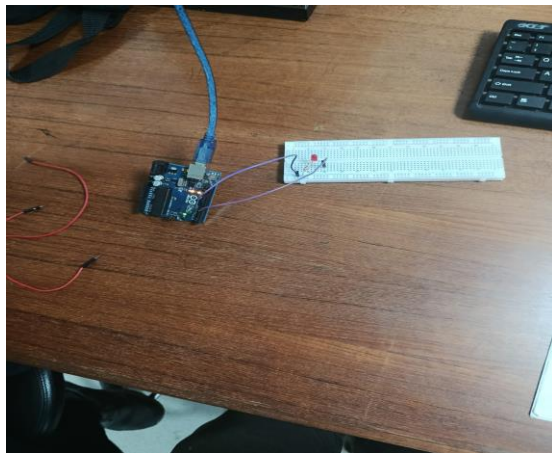


Figure 1: LED Off

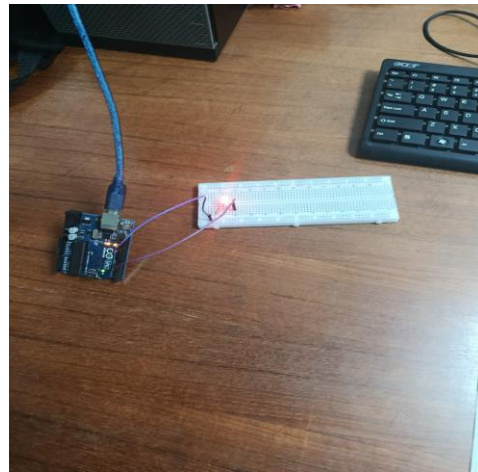


Figure 2: LED On

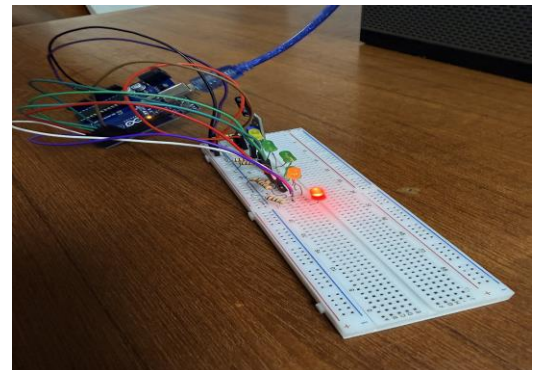
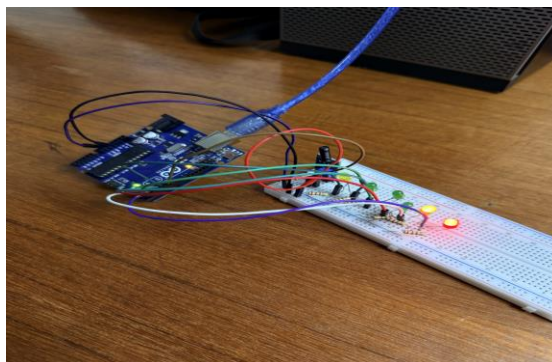


Figure :3

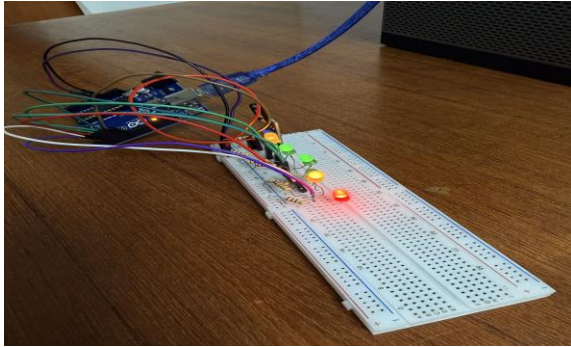
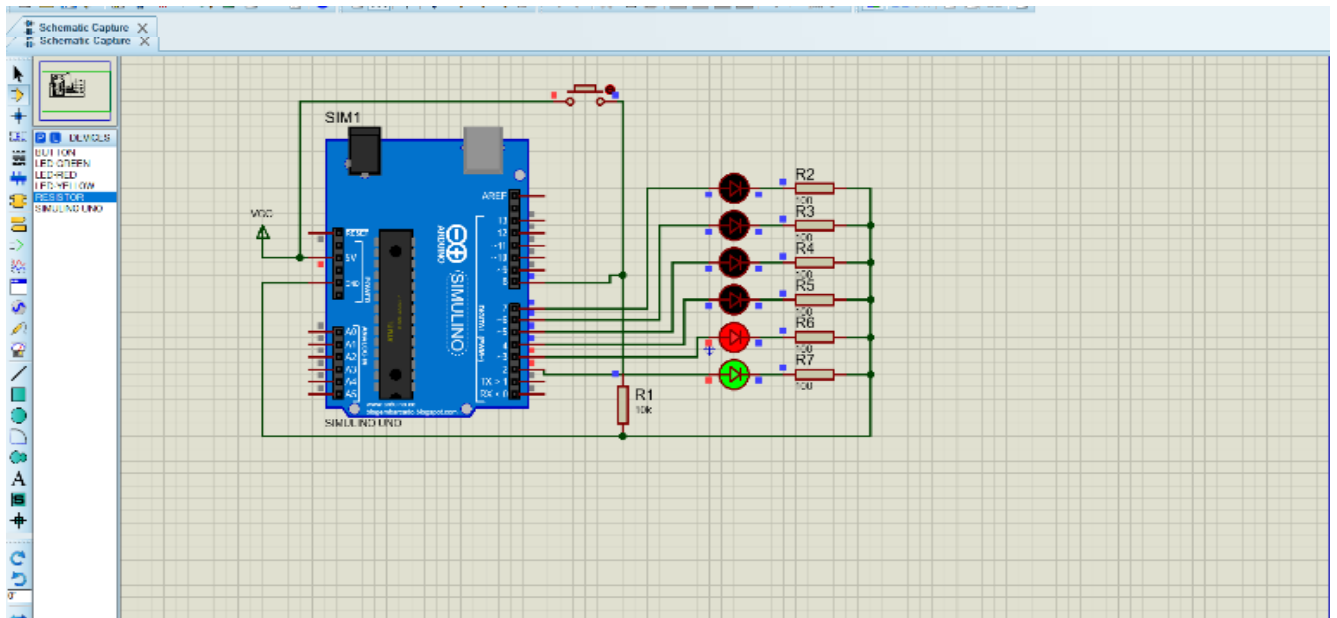


Figure :4

Figure :5

Simulation Output Results:

Figure 7: simulation result for LED Light Pattern Control System when 1 led light on at a time .
Figure 8: simulation result for LED Light Pattern Control System when 2 led light on at a time .



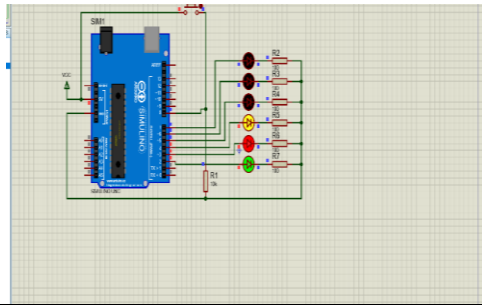


Figure 9: Result from the simulation showing the state where three LEDs are illuminated at once in the lighting sequence

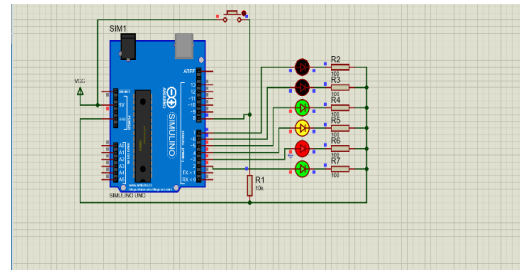


Figure 10: simulation result for LED Light Pattern Control System when 4 led light on at a time .

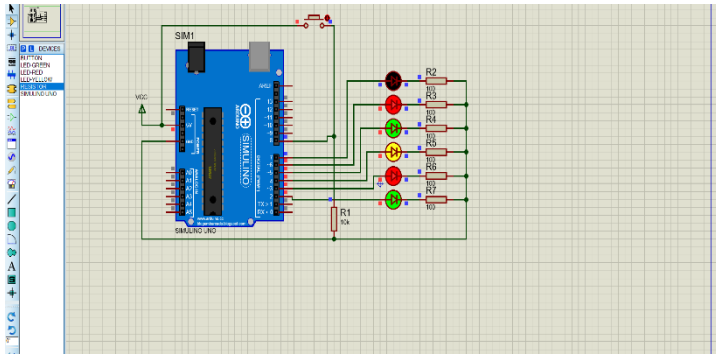


Fig:11

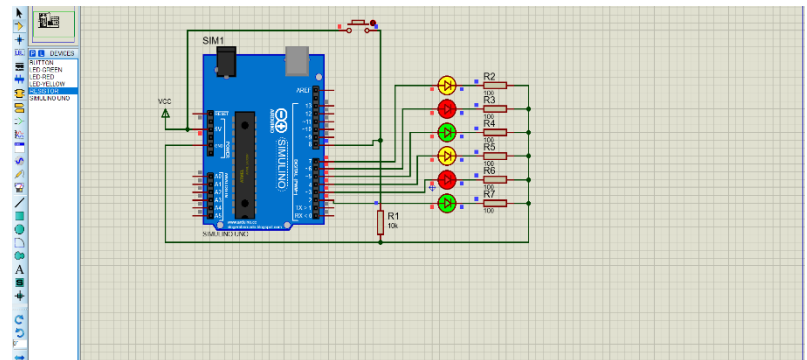


Fig:12

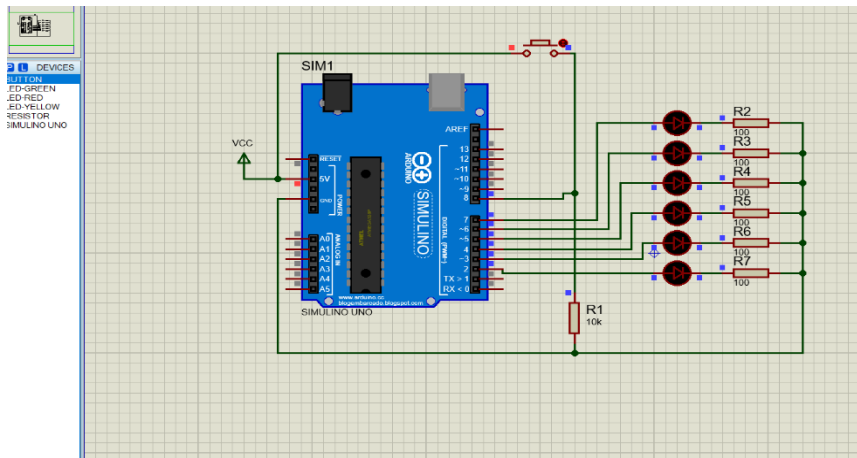


Figure 13: Simulation result for LED Light Pattern Control System when all led light are off .

Simulation Explanation:

The simulation was carried out using Proteus software to validate the functionality of the LED light pattern control system programmed with Arduino using the `millis()` function. The primary objective was to observe if the timing and switching logic performed as expected in a virtual hardware environment.

1. Circuit Design

The circuit was constructed in the Proteus schematic editor. It included an Arduino Uno board, six LEDs, and a push button. The LEDs were connected to digital pins (2 to 7), and the button was connected to pin 8. All connections followed the logic for a non-blocking digital timer using `millis()`, ensuring accurate control over LED lighting sequences without using the `delay()` function.

2. Flowchart Development:

Before implementing the code, a flowchart was drawn to outline the logical steps of the LED lighting process. This helped visualize how each LED would activate sequentially and how the switch input would reset the system. The flowchart served as a reference for writing and debugging the Arduino code.

3. Running the Simulation:

After designing the circuit and coding the logic, the simulation was run in Proteus. The behavior of each LED was monitored to verify that:

- LEDs turned ON one by one at their respective time intervals.
 - Pressing the switch stopped and reset the sequence properly.
- The virtual environment confirmed that the system functioned as intended, demonstrating correct LED transitions and reset behaviour on input detection.

Answers to the Questions in the Lab Manual:

1) Include all codes and scripts in the lab report following the lab report writing template.

Ans:

Here is the code of our experiment with proper explanation:

```

const int SwitchPin = 8;
unsigned long PreviousTime = 0;
int SwitchState = 0;
int PrevSwitchState = 0;
int led = 2;
long interval = 3000; // for 1 min = 20,000 ms delay
void setup()
{
  for (int x = 2; x < 8; x++)
  {
    pinMode(x, OUTPUT);
  }
  pinMode(SwitchPin, INPUT);
}
void loop()
{
  unsigned long CurrentTime = millis();
  if (CurrentTime - PreviousTime > interval)
  {
    PreviousTime = CurrentTime;
    digitalWrite(led, HIGH);
    led++;
    if (led == 7)
    { }
  }
  SwitchState = digitalRead(SwitchPin);
  if (SwitchState != PrevSwitchState)
  {
    for (int x = 2; x < 8; x++)
    {
      digitalWrite(x, LOW);
    }
    led = 2;
    PreviousTime = CurrentTime;
  }
  PrevSwitchState = SwitchState;
}

```

1) Code Explanation:

Already provided above (your long explanation was good and correct). Minor change suggested: "LEDs 2 to 7" instead of just "2 to 6" because 6 LEDs are from pin 2 to 7.

2) Show output/results with captions:

Mentioned to be already included in your **Experimental Output Result** section. No changes needed.

3) Configure with ID-based delay: ID → XY-PQABC-Z = 23-50834-1

So:

- **P = 5 s**

-Q = 0 → Use Z = 1 s

-A = 8 s

-B = 3 s

-C = 4 s

-Z = 1 s

Thus, delays are:

--LED1 = 5 s

-LED2 = 1 s

-LED3 = 8 s

-LED4 = 3 s

-LED5 = 4 s

-LED6 = 1 s

Final Corrected Arduino Code:

```
cpp
CopyEdit
// Define LED pins
const int LED1 = 2;
const int LED2 = 3;
const int LED3 = 4;
const int LED4 = 5;
const int LED5 = 6;
const int LED6 = 7;
const int SwitchPin = 8; // Switch pin

// Delays in seconds based on ID (23-50834-1)
const int delay1 = 5;
const int delay2 = 1; // Q = 0 → use Z = 1
const int delay3 = 8;
const int delay4 = 3;
const int delay5 = 4;
const int delay6 = 1;

// Convert to milliseconds
const int delay1_ms = delay1 * 1000;
const int delay2_ms = delay2 * 1000;
const int delay3_ms = delay3 * 1000;
const int delay4_ms = delay4 * 1000;
const int delay5_ms = delay5 * 1000;
const int delay6_ms = delay6 * 1000;

void setup() {
```

```

pinMode(LED1, OUTPUT);
pinMode(LED2, OUTPUT);
pinMode(LED3, OUTPUT);
pinMode(LED4, OUTPUT);
pinMode(LED5, OUTPUT);
pinMode(LED6, OUTPUT);
pinMode(SwitchPin, INPUT);
}

void loop() {
  int SwitchState = digitalRead(SwitchPin);

  if (SwitchState == HIGH) {
    // Reset all LEDs
    digitalWrite(LED1, LOW);
    digitalWrite(LED2, LOW);
    digitalWrite(LED3, LOW);
    digitalWrite(LED4, LOW);
    digitalWrite(LED5, LOW);
    digitalWrite(LED6, LOW);
    delay(200); // Debounce delay
    return; // Exit loop early to reset
  }

  digitalWrite(LED1, HIGH); delay(delay1_ms);
  digitalWrite(LED2, HIGH); delay(delay2_ms);
  digitalWrite(LED3, HIGH); delay(delay3_ms);
  digitalWrite(LED4, HIGH); delay(delay4_ms);
  digitalWrite(LED5, HIGH); delay(delay5_ms);
  digitalWrite(LED6, HIGH); delay(delay6_ms);
}

```

Hardware Output Results:



Fig 14:

Simulation for above code:

Figure 16: Simulation output of the LED Light Pattern Control System where one LED remains active at a time.

Figure 17: Simulation output of the LED Light Pattern Control System where two LEDs remain active simultaneously.

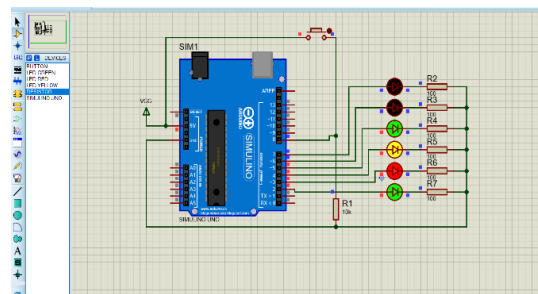
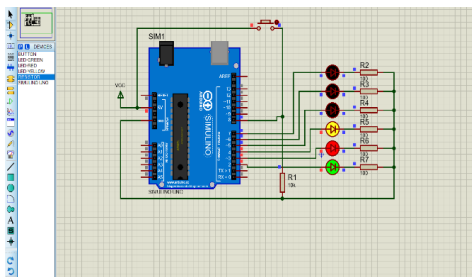
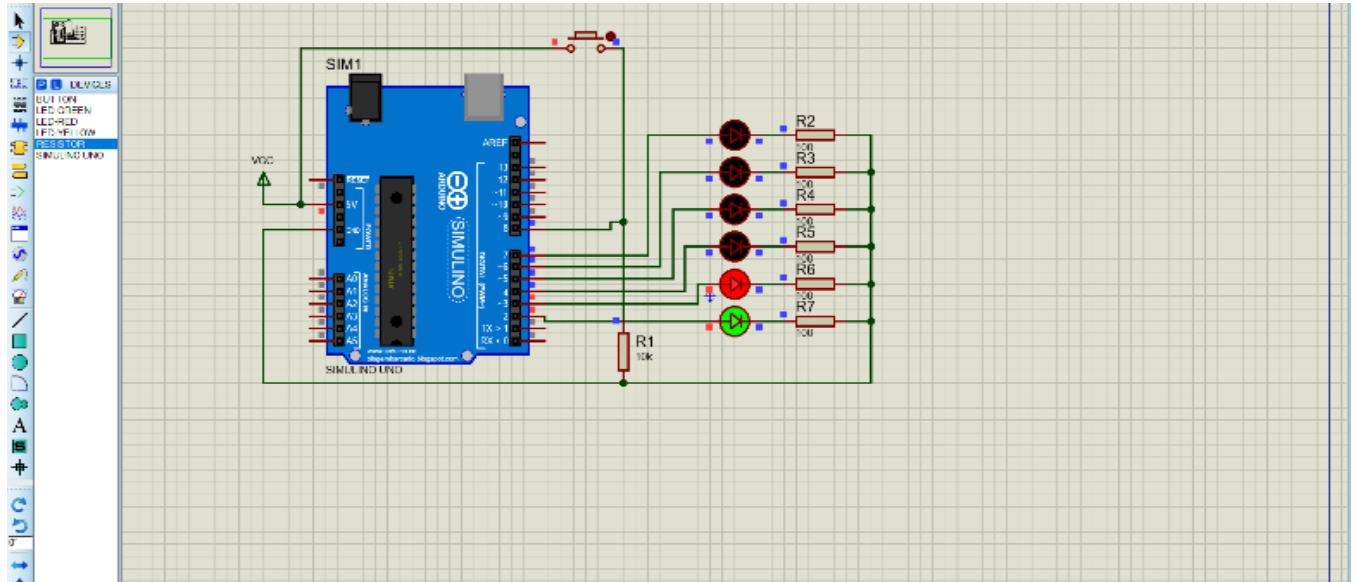


Figure 17: The simulation shows that three LEDs light up simultaneously in a controlled sequence. Each group of LEDs stays on for the set time before switching smoothly to the next group.

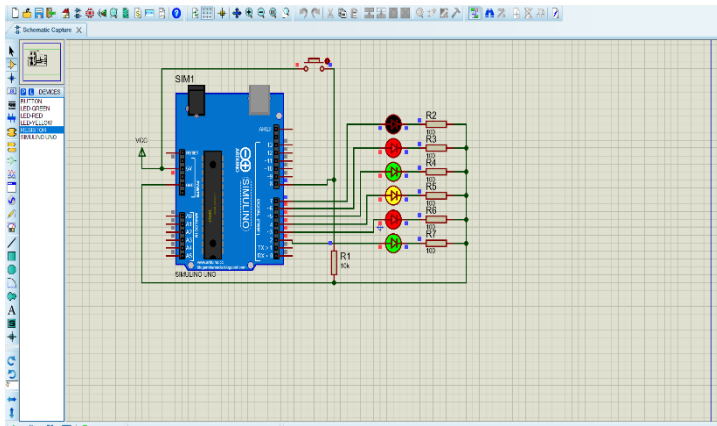


Figure 18: The simulation confirms that five LEDs illuminate together in a timed coordinated pattern.

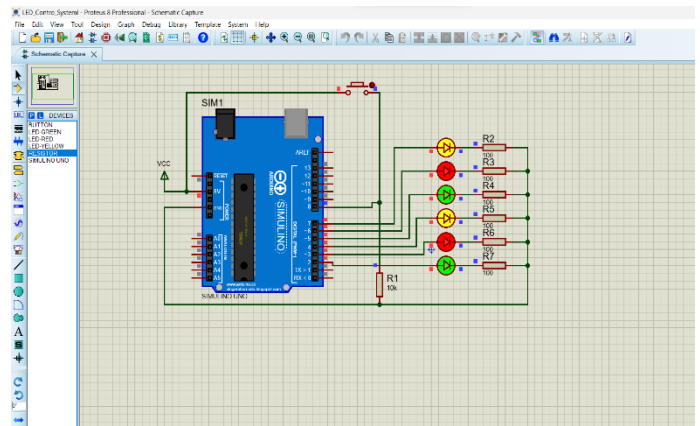


Fig 19: The simulation demonstrates six LEDs lighting up simultaneously in a synchronized sequence.

Discussion and Conclusion:

In this experiment, a basic digital timer system was created using the `millis()` function. Insights about the use of `millis()` were gathered from various resources, including online research and study materials. Essential information and findings were compiled to assist in future projects and implementations. Important details regarding the `millis()` function were also documented for later reference.

The `millis()` function was employed to monitor elapsed time, allowing the program to advance to the next LED once the set interval was reached. It was observed that `millis()` is a built-in Arduino library function and serves as a practical alternative to the `delay()` function. Unlike `delay()`, which pauses the entire program, `millis()` enables other code segments to run concurrently during the waiting period.

Through different observations and sources, certain limitations of the `millis()` function were identified. One challenge highlighted was the difficulty in maintaining precise timing. Additional drawbacks were also noted during further examination. Following the identified logic and approach, the digital timer system was assembled using resistors, LEDs, a tilt switch, connecting wires, an Arduino UNO board, and other necessary components. The code to operate the digital timer was written based on the gathered knowledge of the `millis()` function and timer operation principles. The system's performance and results were recorded for further evaluation.

Simulation results were analysed and found to be satisfactory. For further investigation, an alternative version of the digital timer was developed with varied delay intervals.